



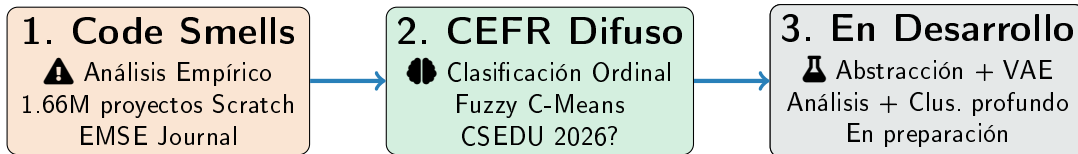
El camino hacia una escala comprensible del código

Un Recorrido por mi Investigación Doctoral

Análisis Empírico de Calidad de Código en Scratch

Clasificación Difusa con FCM y VAE para Habilidades de Programación
y Abstracción Procedimental

Ricardo Hidalgo Aragón · +FuzzyMAD 2025 · Diciembre 2025



≡ Temática Unificadora

Calidad y evaluación de código en educación en programación

Desde el análisis descriptivo hasta la clasificación automática con ML|DL pasando por el análisis masivo de información no estructurada.

Hito 1: Code Smells en Scratch

? La Pregunta Central

¿Las habilidades de Pensamiento Computacional (CT) influyen en la calidad del código?

Motivación:

- En la era de los LLMs, programar = **arquitectar + curar**
- Detectar code smells es crucial
- Necesitamos evidencia empírica masiva

Dataset

1,664,112 proyectos Scratch

Herramientas:

- **Dr. Scratch:** 9 dimensiones CT (0-4)
- **LitterBox:** 34 code smells en 8 familias

Objetivo

Entender la relación entre habilidades CT y presencia/densidad de code smells

✓ RQ1: Relación CT-Smells

Mayor CT → Menor densidad de smells
PERO: 94.8 % mediado por tamaño del proyecto

! La Paradoja de la Ambición

Programadores hábiles crean proyectos **más grandes y complejos**

- Más oportunidades para smells
- La calidad mejora *a pesar* de la escala

⚙ RQ2: Dimensiones Específicas

Abstracción y Lógica: Protectoras ($OR \leq 1$)

Sincronización y Paralelismo: Aumentan complejidad ($OR \approx 1.2-1.4$)

⚖ RQ3: Moderación Contextual

En **coding challenges**:

Efectos de moderación hasta $\Delta\rho = +0,37$

17.1 % de smells son decisiones de diseño justificadas

1. Déficits de Habilidad (esperado)

- Baja Abstracción → Alta Duplicación
- Remediable con instrucción

2. Deuda Técnica Mediada por Ambición

- 94.8 % de mediación por tamaño
- Patrón de reversión en Q4 (proyectos más grandes)
- → La Paradoja de la Ambición

3. Restricciones Intrínsecas de la Tarea

- Contextos de coding challenges: De *mal olor* a solución pragmática.
- Efectos de moderación hasta $\Delta\rho = +0,37$
- Variable Scope "viola" sistemáticamente las mejores prácticas
- → ¡El Contexto Importa!

Implicaciones: De *¿Funciona?* a *¿Puede Evolucionar?*

DEJAR de Hacer

- × Conteos absolutos de smells
- × Políticas de "tolerancia cero"
- × Penalizar smells sensibles al contexto

EMPEZAR a Hacer

- ✓ Métricas basadas en densidad
- ✓ Evaluación consciente del contexto
- ✓ Normalizar el refactoring

Para la Era de LLMs

Estudiantes sin conciencia de calidad:

- Aceptan smells generados por IA
- No pueden guiar refactorings
- Perpetúan patrones malos

Estudiantes con alfabetización en mantenimiento:

- Reconocen patrones problemáticos
- Refactorizan estratégicamente

Alfabetización en calidad = habilidad de supervivencia profesional

Publicación

ICSME 2025 (Core A)

Registered Report

Preprint:

<https://arxiv.org/abs/2507.17314>

Enviado a **EMSE** (Q1)

Datos Clave

- 1,664,112 proyectos analizados
- 144 modelos GLM ($9 \times 8 \times 2$)
- FDR-corrected, $q < 0,01$
- Validación cualitativa ($n=387$)
- Paquete de replicación (MIT)

Contribución Teórica

Primera evidencia empírica a gran escala de que las habilidades CT predicen la calidad del código, pero con tres mecanismos distintos (habilidad, ambición, contexto)



Hito 2: Clasificación Difusa CEFR

El Problema: ¿Cómo Medimos las Habilidades de Programación?

! Situación Actual

- Rúbricas **coarse-grained** y subjetivas
- Modelos ML **opacos** (cajas negras)
- Sin **estándar universal** de niveles
- Difícil **comunicar** el nivel real

💡 La Propuesta

Adaptar el **CEFR** (A1→C2) usado en idiomas a las habilidades de programación



¿Por Qué Lógica Difusa? El Aprendizaje No Es Binario

? El Dilema

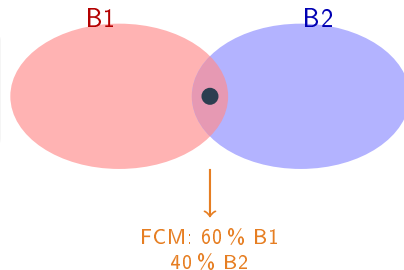
Un estudiante con $[3, 2, 4, 1, 3, 2, 4, 0, 3]$...
¿Es B1 o B2? ¿O *un poco de ambos*?

Clasificación Rígida (K-Means):

- × Fuerza pertenencia a un solo cluster
- × Pierde información de **frontera**

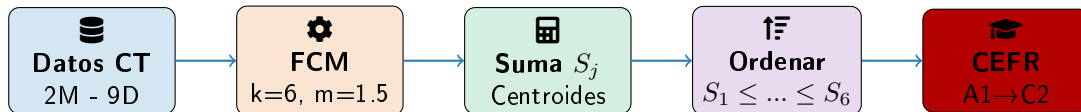
Clasificación Difusa (FCM):

- ✓ Grados de pertenencia: $u_{ij} \in [0, 1]$
- ✓ Captura **solapamiento** natural
- ✓ Respeta la **gradualidad** del aprendizaje



Clave: El aprendizaje es un *continuo*,
no una serie de saltos discretos

Método: De FCM a Escala Ordinal CEFR



Insight Principal

Al trabajar con **variables ordinales**, la suma de valores de centroides $S_j = \sum_{d=1}^9 v_{jd}$ refleja naturalmente el **nivel de competencia**



Innovación

Tres métricas avanzadas:

- Detección de transiciones
- Score continuo (1-6)
- Certeza (entropía Shannon)

✓ Tests Estadísticos

- **Kruskal-Wallis:** $H = 14,378$, $p \approx 0$
- **Mann-Whitney U:** Todos pares significativos ($p < 0,001$)
- **Kendall τ :** 1.000 ($p = 0,0028$)
- **Spearman ρ :** 1.000 ($p < 0,001$)

↑ Progresión Confirmada

$C(A1) < C(A2) < C(B1) < C(B2) < C(C1) < C(C2)$

Correlación **perfecta** entre niveles CEFR y medianas CT

Comparación	p-valor
A1 vs A2	$5,7 \times 10^{-163}$
A2 vs B1	≈ 0
B1 vs B2	$1,1 \times 10^{-107}$
B2 vs C1	≈ 0
C1 vs C2	≈ 0

Resultado: Cada nivel CEFR es *significativamente distinto*



Aplicaciones

Educación:

- Autoevaluación
- Seguimiento de progresión
- Certificaciones estandarizadas

Industria:

- Screening automatizado
- Rutas de formación

Difusión

En preparación para:

CSEDU 2026

Dataset: 2M proyectos

Código: Disponible (MIT)



Contribución

Framework interpretable y universal para evaluar habilidades de programación con estándar internacional (CEFR)

Hito 3: Investigación en Curso



Arquitectura Propuesta

VAE (Encoder-Decoder):

- Encoder: 9D $\rightarrow$ latente 1D
- Decoder: 1D $\rightarrow$ 9D
- Captura estructura ordinal

Clustering en espacio latente:

- K-Means
- Gaussian Mixture Models
- Ordenamiento por centroides

Función de pérdida:

- **MSE**: reconstrucción fiel
- **KL**: espacio latente ordenado y suave



Ventajas sobre FCM

- **Aprendizaje profundo** de patrones no lineales
- **1D interpretable**: visualización directa
- **Regularización KL**: estructura ordinal suave
- **Generativo**: reconstrucción y validación



Estado Actual

- Arquitectura optimizada con AG
- Validación en 2M observaciones
- Comparación FCM vs VAE+KM vs VAE+GMM

Métricas de Entrenamiento

Mejor arquitectura (GA):

Hidden layers: [17]

Pérdidas finales:

- Train Loss: 1.646
- Val Loss: 1.645
- MSE: ~ 1.13
- KL: ~ 0.51

Validación Ordinal

Tests estadísticos (GMM):

- Kruskal-Wallis: $p \approx 0$
- Kendall τ : 0.77-0.93
- Spearman ρ : 0.85-0.97

Comparación:

- FCM: más directo, interpretable
- VAE+GMM: mayor flexibilidad, potencial predictivo

Próximos Pasos

- 1) Análisis comparativo exhaustivo FCM vs VAE
- 2) Visualizaciones del espacio latente
- 3) Validación con expertos

? El Problema

Clonación de código vs. Custom Blocks

Los usuarios clonan donde deberían abstraer

Escala del Análisis

2.5 millones de proyectos Scratch

Detección de:

- Clones Type I y II
- Intra-sprite e inter-sprite
- Custom Blocks y parámetros

+ Hallazgos Preliminares

Clonación:

- 10 % proyectos: clones intra-sprite
- 16 % proyectos: clones inter-sprite

Custom Blocks:

- 81.8 % son *alias* (sin parámetros)
- CBs con parámetros: menos clonación
- Muchos CBs con 1 bloque o sin llamadas

Hipótesis: Limitaciones de Scratch + falta de énfasis educativo en abstracción

Roadmap: ¿Hacia Dónde Vamos?



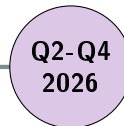
Code Smells Preprint
ICSME 2025
✓



Empirical Code Smells
EMSE
✓



Clasificación Difusa
En escritura
✍



VAE Ordinal
+ Abstracción
En desarrollo
🧪

✓ Completado

- Code Smells: ICSME + EMSE
- CEFR (75 %): CSEDU?

⌚ En Proceso

- Abstracción: experimentación + escritura
- VAE: experimentación + escritura

Conclusiones: Un Recorrido de Análisis a Clasificación



Code Smells

Contexto y ambición importan



CEFR Difuso

Escala ordinal interpretable



Abstracción

Déficit educativo evidenciado



VAE Ordinal

Deep learning para clasificación

“ Hilo Conductor

De la descripción empírica masiva a la clasificación automática interpretable.

¡Gracias!

¿Preguntas?

✉ r.hidalgoa.2024@alumnos.urjc.es

🐙 github.com/rharagon